

Lecture 5

Regularization

Regularization

A central problem in machine learning is how to make an algorithm that will perform well not just on the training data, but also on new inputs.

Most machine learning tasks are estimation of a function $\hat{f}(x)$ parameterized by a vector of parameters $\mathbf{w}$.

Classical Regularization: Parameter Norm Penalty

Most classical regularization approaches are based on limiting the capacity of models, by adding a parameter norm penalty $\Omega(\mathbf{w})$ to the objective function J .

$$J(\mathbf{w}; X, y) = J(\mathbf{w}; X, y) + \alpha\Omega(\mathbf{w})$$

L^2 Parameter Regularization, weight decay

L^2 parameter norm penalty commonly known as weight decay.

Regularization term $\Omega(\mathbf{w}) = \frac{1}{2} \|\mathbf{w}\|_2^2$

Gradient of the total objective function:

$$\nabla_{\mathbf{w}} \tilde{J}(\mathbf{w}; X, y) = \alpha \mathbf{w} + \nabla_{\mathbf{w}} J(\mathbf{w}; X, y)$$

$$\mathbf{w} := \mathbf{w} - \epsilon(\alpha \mathbf{w} + \nabla_{\mathbf{w}} J(\mathbf{w}; X, y))$$

Considering a quadratic approximation to the objective function:

$$\hat{J}(\mathbf{w}) = J(\mathbf{w}^*) + \frac{1}{2}(\mathbf{w} - \mathbf{w}^*)^T H(\mathbf{w} - \mathbf{w}^*)$$

$$\nabla_{\mathbf{w}} \hat{J}(\mathbf{w}) = H(\mathbf{w} - \mathbf{w}^*)$$

$$\alpha w + H(w - w^*) = 0$$

$$(H + \alpha I)w = Hw^*$$

$$\tilde{w} = (H + \alpha I)^{-1} Hw^*.$$

what happens as α grows?

$$H = Q\Lambda Q^T$$

$$\tilde{w} = (Q\Lambda Q^T + \alpha I)^{-1} Q\Lambda Q^T w^*$$

$$= [Q(\Lambda + \alpha I)Q^T]^{-1} Q\Lambda Q^T w^*$$

$$= Q(\Lambda + \alpha I)^{-1} \Lambda Q^T w^*,$$

$$Q^T \tilde{w} = (\Lambda + \alpha I)^{-1} \Lambda Q^T w^*.$$

The effect of weight decay is to rescale the coefficients of eigenvectors. The i th component is rescaled by a factor of $\frac{\lambda_i}{\lambda_i + \alpha}$.

- ▶ If $\lambda_i \gg \alpha$, the effect of regularization is relatively small.
- ▶ Components with $\lambda_i \ll \alpha$ will be shrunk to have nearly zero magnitude.

Directions along which the parameters contribute significantly to reducing the objective function are preserved a small eigenvalue of the Hessian tell us that movement in this direction will not significantly increase the gradient **effective number of parameters**, defined to be

$$\gamma = \sum_i \frac{\lambda_i}{\lambda_i + \alpha}.$$

As α is increased, the effective number of parameters decreases.

Dataset Augmentation

The best way to make a machine learning model generalize better is to train it on more data.

Dataset Augmentation

The best way to make a machine learning model generalize better is to train it on more data.

The amount of data we have is limited.

Dataset Augmentation

The best way to make a machine learning model generalize better is to train it on more data.

The amount of data we have is limited.

Create fake data and add it to the training set.

Dataset Augmentation

The best way to make a machine learning model generalize better is to train it on more data.

The amount of data we have is limited.

Create fake data and add it to the training set.

Not applicable to all tasks.

Dataset Augmentation

The best way to make a machine learning model generalize better is to train it on more data.

The amount of data we have is limited.

Create fake data and add it to the training set.

Not applicable to all tasks.

For example, it is difficult to generate new fake data for a density estimation task unless we have already solved the density estimation problem.

Dataset Augmentation

Operations like translating the training images a few pixels in each direction can often greatly improve generalization.

One way to improve the robustness of neural networks is simply to train them with random noise applied to their inputs.

This same approach also works when the noise is applied to the hidden units, which can be seen as doing dataset augmentation at multiple levels of abstraction.

Noise injection

Two ways that noise can be used as part of a regularization strategy.

Adding noise to the input.

This can be interpreted simply as form of dataset augmentation.

Can also interpret it as being equivalent to more traditional forms of regularization.

Noise injection

Two ways that noise can be used as part of a regularization strategy.

Adding noise to the input.

This can be interpreted simply as form of dataset augmentation.

Can also interpret it as being equivalent to more traditional forms of regularization.

Adding it to the weights.

This technique has been used primarily in the context of recurrent neural networks (Jim et al., 1996; Graves, 2011a).

This can be interpreted as a stochastic implementation of a Bayesian inference over the weights.

Small input noise adds a deterministic regularizer

Consider a regression model with squared-error loss:

$$f(\mathbf{x}; \mathbf{w}) = \hat{y}(\mathbf{x})$$

$$J(\mathbf{w}) = E_{(\mathbf{x}, y)} \left[\frac{1}{2} (f(\mathbf{x}; \mathbf{w}) - y)^2 \right]$$

Inject zero-mean noise into the input:

$$\tilde{\mathbf{x}} = \mathbf{x} + \boldsymbol{\varepsilon}, \quad E[\boldsymbol{\varepsilon}] = 0, \quad \text{Cov}(\boldsymbol{\varepsilon}) = \boldsymbol{\Sigma}$$

The noisy training objective becomes:

$$J_{\text{noise}}(\mathbf{w}) = E_{(\mathbf{x}, y, \boldsymbol{\varepsilon})} \left[\frac{1}{2} (f(\mathbf{x} + \boldsymbol{\varepsilon}; \mathbf{w}) - y)^2 \right]$$

For small noise, this is equivalent to adding a penalty that discourages predictions from changing rapidly with the input.

A second-order expansion isolates the relevant terms

Let

$$r = f(x) - y, \quad g = \nabla_x f(x), \quad H = \nabla_x^2 f(x).$$

A second-order Taylor expansion gives:

$$f(x+\epsilon) - y \approx r + g^T \epsilon + \frac{1}{2} \epsilon^T H \epsilon$$

Substituting into the squared loss and keeping terms through order ϵ^2 :

$$2\ell_{\text{noise}} \approx r^2 + 2r g^T \epsilon + r \epsilon^T H \epsilon + (g^T \epsilon)^2$$

Terms containing three or four factors of ϵ are higher order and are dropped.

Averaging over noise reveals two penalties

Use the covariance identities:

$$E[g^T \varepsilon] = 0$$

$$E[\varepsilon^T H \varepsilon] = \text{tr}(H \Sigma)$$

$$E[(g^T \varepsilon)^2] = g^T \Sigma g$$

Therefore, at one training example:

$$E \ell_{\text{noise}} \approx \frac{1}{2} r^2 + \frac{1}{2} r \cdot \text{tr}(H \Sigma) + \frac{1}{2} g^T \Sigma g$$

For isotropic noise, $\Sigma = vI$:

$$J_{\text{noise}} \approx J + \frac{1}{2} v E[(f - y) \text{tr}(H)] + \frac{1}{2} v E[\|\nabla_x f\|^2]$$

The first added term depends on curvature; the second directly penalizes input sensitivity.

Near least squares, only the sensitivity penalty remains

For squared error, the optimal predictor is the conditional mean:

$$f^*(x) = E[y | x]$$

Hence the residual has zero conditional mean:

$$E_{y|x} [f^*(x) - y] = 0$$

Small noise shifts the solution by $O(v)$. Since the curvature term already contains v , its contribution is $O(v^2)$ and can be ignored to first order.

Thus the first-order noisy objective is:

$$J_{\text{noise}} \approx J + \frac{1}{2} E[\nabla_x f(x)^T \Sigma \nabla_x f(x)]$$

Noise injection is therefore a Jacobian, or input-sensitivity, regularizer.

Noise covariance selects the directions we smooth

Decompose the noise covariance into its eigen-directions:

$$\Sigma = U \text{diag}(\sigma_1^2, \dots, \sigma_d^2) U^T$$

Then the sensitivity penalty becomes:

$$\nabla f^T \Sigma \nabla f = \sum_i \sigma_i^2 (u_i^T \nabla f)^2$$

- u_i is an eigenvector: a direction in input space.
- σ_i^2 is the injected noise variance in that direction.
- $u_i^T \nabla f$ is the sensitivity of the prediction along that direction.

Larger noise variance produces stronger smoothing in the corresponding direction—parallel to the eigen-direction interpretation of L^2 regularization.

For linear models, input noise is exactly L^2 weight decay

For a linear predictor:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \quad \nabla_{\mathbf{x}} f = \mathbf{w}, \quad \nabla_{\mathbf{x}}^2 f = 0$$

The expected noisy squared loss is exactly:

$$\begin{aligned} & \mathbb{E} \left[\frac{1}{2} (\mathbf{w}^T (\mathbf{x} + \boldsymbol{\varepsilon}) + b - y)^2 \right] \\ &= \frac{1}{2} (\mathbf{w}^T \mathbf{x} + b - y)^2 + \frac{1}{2} \mathbf{w}^T \boldsymbol{\Sigma} \mathbf{w} \end{aligned}$$

For isotropic noise, $\boldsymbol{\Sigma} = v\mathbf{I}$:

$$J_{\text{noise}} = J + \frac{1}{2} v \|\mathbf{w}\|^2$$

Thus a nonlinear network receives a local Jacobian penalty, while a linear model receives ordinary L^2 weight decay.

Manifold Tangent Classifier

It is assumed that we are trying to classify examples and that examples on the same manifold share the same category.

The classifier should be invariant to the local factors of variation that correspond to movement on the manifold.

Use as nearest-neighbor distance between points x_1 and x_2 the distance between the manifolds M_1 and M_2 to which they respectively belong.

Manifold Tangent Classifier

Approximate M_i by its tangent plane at x_i and measure the distance between the two tangent.

Train a neural net classifier with an extra penalty to make the output $f(x)$ of the neural net locally invariant to known factors of variation (Tangent-Prop algorithm, Simard et al., 1992).

Manifold Tangent Classifier

Require $\frac{\partial f(x)}{\partial x}$ to be orthogonal to the known manifold tangent vectors v_i at x .

Or the directional derivative of f at x in the directions v_i be small.

$$\text{regulaizer} = \lambda \sum_i \left(\frac{\partial f(x)}{\partial x} \cdot v_i \right)^2$$

A recent paper introduces the Manifold Tangent Classifier (Rifai et al., 2011), which eliminates the need to know the tangent vectors a priori.

Early Stopping as Regularization

- Stop training when the validation error **fails to improve for a set number of steps** (patience) or starts to increase.
- **Patience:** Typically 5-10 epochs without improvement before stopping.
- Return the model with the lowest validation error, not the final one after training.
- Early stopping is a widely used regularization technique in deep learning.

Early Stopping as Regularization

- Stop training when the validation error **fails to improve for a set number of steps** (patience) or starts to increase.
- **Patience:** Typically 5-10 epochs without improvement before stopping.
- Return the model with the lowest validation error, not the final one after training.

Example in Keras:

- Early st

```
from keras.callbacks import EarlyStopping
```

```
early_stopping = EarlyStopping(monitor='val_loss', patience=5)
```

Early Stopping as a Form of Regularization

- Take a quadratic approximation of the objective function J around the empirically optimal weight w^* .

$$\hat{J}(w) = J(w^*) + \frac{1}{2}(w - w^*)^T H(w - w^*)$$

$$\nabla_w \hat{J}(w) = H(w - w^*)$$

- Study the trajectory of the parameter vector during training. Initialize w to the origin: $w^{(0)} = 0$.

$$\begin{aligned}
 w^{(\tau)} &= w^{(\tau-1)} - \eta \nabla_w J(w^{(\tau-1)}) \\
 &= w^{\tau-1} - \eta H(w^{(\tau-1)} - w^*)
 \end{aligned}$$

$$w^{(\tau)} - w^* = (I - \eta H)(w^{(\tau-1)} - w^*)$$

$$H : H = Q\Lambda Q$$

$$w^{(\tau)} - w^* = (I - \eta Q\Lambda Q^T)(w^{(\tau-1)} - w^*)$$

$$Q^T(w^{(\tau)} - w^*) = (I - \eta \Lambda)Q^T(w^{(\tau-1)} - w^*)$$

Assume $w^{(0)} = 0$, and that $|1 - \eta\lambda_i| < 1$, we have after τ training updates,

$$Q^T w^{(\tau)} = [I - (I - \eta\Lambda)^\tau] Q^T w^*$$

$$Q^T \tilde{w} = (\Lambda + \alpha I)^{-1} \Lambda Q^T w^*$$

$$Q^T \tilde{w} = [I - (\Lambda + \alpha I)^{-1} \alpha] Q^T w^*$$

$$(I - \eta\Lambda)^\tau = (\Lambda + \alpha I)^{-1} \alpha$$

by taking logs and using the series expansion for $\log(1+x)$, we can conclude that if all λ_i are small (i.e. $\eta\lambda_i \ll 1$ and $\lambda_i/\alpha \ll 1$) then

$$\tau \approx \frac{1}{\eta\alpha},$$

$$\alpha \approx \frac{1}{\tau\eta}.$$

the number of training iterations τ plays a role inversely proportional to the L^2 regularization parameter, and the inverse of $\tau\eta$ plays the role of the weight decay coefficient.

Early stopping is a spectral regularizer

The key idea: gradient descent learns some curvature directions faster than others.

1 Approximate the training loss near its

$$J(w) \approx J(w^*) + \frac{1}{2} (w - w^*)^T H (w - w^*)$$

2 Rotate into the eigenvectors of H

$$H = Q\Lambda Q^T, \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_k), \quad z = Q^T w$$

In the z-coordinates, each direction i evolves independently. We only need to solve one scalar recurrence.

One eigendirection gives the whole derivation

Assume $z^{(0)} = 0$ and choose $0 < \eta\lambda_i < 1$ for a simple monotone trajectory.

Gradient-descent update

$$z_i^{(t)} = z_i^{(t-1)} - \eta\lambda_i (z_i^{(t-1)} - z_i^*)$$

Define the error $e_i^{(t)} = z_i^{(t)} - z_i^*$

$$e_i^{(t)} = (1 - \eta\lambda_i)e_i^{(t-1)} \Rightarrow e_i^{(t)} = (1 - \eta\lambda_i)^t e_i^{(0)}$$

Substitute $e_i^{(0)} = -z_i^*$

$$z_i^{(t)} = [1 - (1 - \eta\lambda_i)^t] z_i^*$$

Stopping at time t applies a curvature-dependent filter

Define the fraction of direction i that has been learned by time t .

$$g_t(\lambda) = 1 - (1 - \eta\lambda)^t, \quad z_i^{(t)} = g_t(\lambda_i) z_i^*$$

LARGE CURVATURE λ_i

$(1 - \eta\lambda_i)^t$ decays quickly

The direction is learned early.



mostly learned

SMALL CURVATURE λ_i

$(1 - \eta\lambda_i)^t$ stays near 1

The direction is still suppressed.



barely learned

Early stopping regularizes by attenuating the slow, low-curvature directions.

L^2 regularization produces a similar spectral shrinkage

Add an L^2 penalty and solve the same quadratic model exactly.

$$\text{minimize } J(w) + (\alpha/2)\|w\|^2$$

$$z_i(\text{ridge}) = [\lambda_i / (\lambda_i + \alpha)] z_i^*$$

Direction	Early stopping	L^2 / ridge
large λ_i	$g_t(\lambda_i) \approx 1$	$\lambda_i / (\lambda_i + \alpha) \approx 1$
small λ_i	$g_t(\lambda_i)$ is small	$\lambda_i / (\lambda_i + \alpha)$ is small

Both methods preserve high-curvature signal and shrink low-curvature directions.

Training time acts like an inverse regularization strength

The correspondence is approximate: the two filters have different shapes, but similar transition scales.

1 Small-step approximation

$$(1 - \eta\lambda)^t \approx \exp(-\eta\lambda t)$$

$$g_t(\lambda) \approx 1 - \exp(-\eta\lambda t)$$

2 Compare where each filter turns on

early stopping: $\eta\lambda t \approx 1 \Rightarrow \lambda \approx 1/(\eta t)$

ridge: $\lambda \approx \alpha$

effective $\alpha \approx 1/(\eta t)$

The stopping time is the regularization knob

Validation chooses how much of the slow spectrum the model is allowed to learn.

Stop earlier

$t \downarrow \Rightarrow \text{effective } \alpha \uparrow$

stronger regularization

Train longer

$t \uparrow \Rightarrow \text{effective } \alpha \downarrow$

weaker regularization

Use validation

Select the checkpoint with the lowest validation loss—not the final iterate.

Practical rule

Train $\rightarrow$ monitor validation loss $\rightarrow$ stop after patience $\rightarrow$ restore the best checkpoint.

What is exact—and what is intuition?

The derivation is exact for a quadratic loss, full-batch gradient descent, constant η , and zero initialization. For neural networks it is a useful local/spectral explanation, not an exact equality to one fixed L^2 penalty.

Parameter Tying and Parameter Sharing

Sometimes we may need other ways to express our prior knowledge about suitable values of the model parameters.

Certain parameters should be close to one another

Parameter Tying and Parameter Sharing

Consider model a with parameters $w^{(a)}$ and model b with parameters $w^{(b)}$. Suppose the two models map the input to two different, but related outputs: $\hat{y}_a = f(w^{(a)}, x)$ and $\hat{y}_b = g(w^{(b)}, x)$.

Imagine that the tasks are similar enough (perhaps with similar input and output distributions) that we believe the model parameters should be close to each other, i.e. $\forall_i, w_i^{(a)}$ should be close to $w^{(b)}$

use a parameter norm penalty of the form: $\Omega(w^{(a)}, w^{(b)}) = \|w^{(a)} - w^{(b)}\|_2^2$.

Label Smoothing

- **Definition:**
 - Label Smoothing is a regularization technique that modifies the one-hot encoded labels to reduce overconfidence in neural network predictions.
 - Instead of assigning a probability of 1 to the correct class and 0 to all others, label smoothing assigns a slightly lower probability to the correct class and distributes the remaining probability across the other classes.
- **Purpose:**
 - To prevent the model from becoming overly confident, which can lead to poor generalization.
 - Helps in mitigating overfitting, especially in scenarios where the model might memorize the training data.



Mathematical Formulation

- For a given label y and a smoothing parameter α , the new label becomes:

$$y_{\text{smooth}} = (1 - \alpha) \cdot y + \frac{\alpha}{k}$$

- α is typically a small value (e.g., 0.1).
- k is the total number of classes.
- Example:
 - For a 3-class problem with true label $y = [0, 1, 0]$ and $\alpha = 0.1$:

$$y_{\text{smooth}} = [0.033, 0.933, 0.033]$$

Label Smoothing

- **Benefits:**
 - **Prevents Overconfidence:** Softens the predictions, making the model less certain and more adaptable.
 - **Improves Generalization:** Reduces overfitting by encouraging the model to be more flexible and less biased toward any single class.
 - **Enhances Calibration:** Models trained with label smoothing tend to produce more calibrated probability estimates, which is important for downstream tasks like uncertainty estimation.

Bagging and Other Ensemble Methods

Bagging (short for bootstrap aggregating) is a technique for reducing generalization error by combining several models (Breiman, 1994).

Train several different models separately, then have all of the models vote on the output for test examples.

This is an example of a general strategy in machine learning called model averaging.

Techniques employing this strategy are known as ensemble methods.

Bagging and Other Ensemble Methods

Consider for example a set of k regression models.

Suppose that each model makes an error ϵ_i on each example.

Suppose the errors drawn from a zero-mean multivariate normal distribution with:

variances $\mathbb{E}[\epsilon_i^2] = \nu$

and covariances $\mathbb{E}[\epsilon_i \epsilon_j] = c$.

Bagging and Other Ensemble Methods

Then the error made by the average prediction of all the ensemble models is:

$$\frac{1}{k} \sum_i \epsilon_i$$

The expected squared error of the ensemble predictor is:

$$\begin{aligned} & \mathbb{E} \left[\frac{1}{k} \sum_i \epsilon_i^2 \right] \\ &= \frac{1}{k^2} \mathbb{E} \left[\sum_i \left(\epsilon_i^2 + \sum_{j \neq i} \epsilon_i \epsilon_j \right) \right] \\ &= \frac{1}{k} \nu + \frac{k-1}{k} c \end{aligned}$$

Dropout

- Dropout is one of the techniques for preventing overfitting in deep neural network which contains a large number of parameters.

Original Paper

- Title:
 - Dropout: A Simple Way to Prevent Neural Networks from Overfitting (2014).
- Authors:
 - Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, Ruslan Salakhutdinov
- Organization:
 - Department of Computer Science, University of Toronto
- URL:
 - <https://www.cs.toronto.edu/~hinton/absps/JMLRdropout.pdf>

Overview

- The key idea is to randomly drop units from the neural network during training.
- During training, dropout samples from number of different “thinned” network.
- At test time, we approximate the effect of averaging the predictions of all these thinned networks.

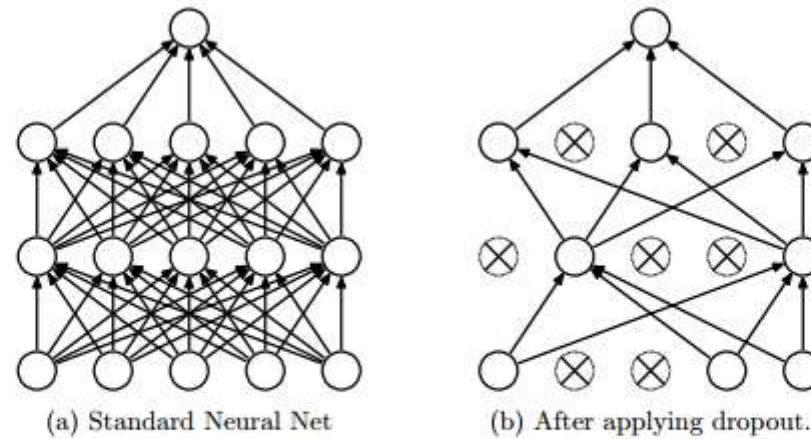


Figure 1: Dropout Neural Net Model. **Left:** A standard neural net with 2 hidden layers. **Right:** An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

Model

- ▶ Consider a neural network with L hidden layer.
- ▶ Let $\mathbf{z}^{(l)}$ denote the vector inputs into layer l
- ▶ $\mathbf{y}^{(l)}$ denote the vector of outputs from layer l .
- ▶ $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are the weights and biases at layer l . With dropout, the feed-forward operation becomes:

Model

$$r_j^{(l)} \sim \text{Bernoulli}(p)$$

$\tilde{\mathbf{y}}^{(l)} = \mathbf{r}^{(l)} \odot \mathbf{y}^{(l)}$, here $\odot$ denotes an element-wise product.

$$z_i^{(l+1)} = \mathbf{w}_i^{(l+1)} \tilde{\mathbf{y}}^{(l)} + b_i^{(l+1)}$$

$y_i^{(l+1)} = f(z_i^{(l+1)})$, where f is the activation function.

For any layer l , $\mathbf{r}^{(l)}$ is a vector of independent Bernoulli random variables each of which has probability of p of being 1. $\tilde{\mathbf{y}}$ is the input after we drop some hidden units. The rest of the model remains the same as the regular feed-forward neural network.

Training

- ▶ Dropout neural network can be trained using **stochastic gradient descent**.
- ▶ The only difference here is that we only back propagate on each thinned network.
- ▶ The gradient for each parameter are averaged over the training cases in each mini-batch.

Test Time

- ▶ use a single neural net without dropout.
- ▶ If a unit is retained with probability p during training, the outgoing weights of that unit are multiplied by p at test time.

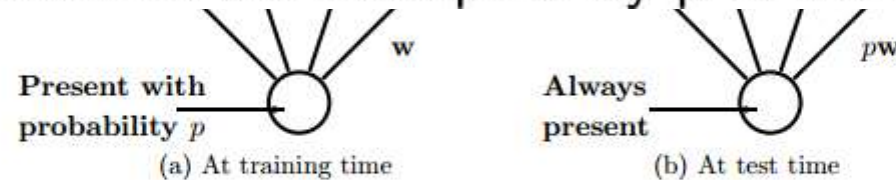


Figure 2: **Left:** A unit at training time that is present with probability p and is connected to units in the next layer with weights w . **Right:** At test time, the unit is always present and the weights are multiplied by p . The output at test time is same as the expected output at training time.

Applying dropout to linear regression

Let $X \in \mathbb{R}^{n \times d}$ be a data matrix of n data points. $y \in \mathbb{R}^n$ be a vector of targets. Linear regression tries to find a $w \in \mathbb{R}^d$ that maximizes:

$$\|y - Xw\|^2$$

When the input X is dropped out such that any input dimension is retained with probability p , the input can be expressed as $R * X$ where $R \in \{0, 1\}^{n \times d}$ is a random matrix with $R_{ij} \sim \text{Bernoulli}(p)$ and $*$ denotes element-wise product. Marginalizing the noise, the objective function becomes:

$$\min_w \mathbb{E}_{R \sim \text{Bernoulli}(p)} [\|y - (R * X)w\|^2]$$

Applying dropout to linear regression

This reduce to:

$$\min_{\mathbf{w}} \| \mathbf{y} - pX\mathbf{w} \|^2 + p(1 - p) \| \Gamma\mathbf{w} \|^2$$

where $\Gamma = (\text{diag}(X^T X))^{\frac{1}{2}}$. Therefore, dropout with linear regression is equivalent to ridge regression with a particular form for Γ . This form of Γ essentially scales the weight cost for weight w_i by the standard deviation of the i^{th} dimension of the data. If a particular data dimension varies a lot, the regularizer tries to squeeze its weight more.

Applying dropout to linear regression

The linear regression model can be represented as:

$$\mathbf{y} = \mathbf{w}^T \mathbf{X}$$

The objective function $J(\mathbf{w})$ in ridge regression is:

$$J(\mathbf{w}) = ||\mathbf{y} - \mathbf{w}^T \mathbf{X}||^2 + \lambda ||\mathbf{w}||^2$$

Applying dropout to linear regression

When dropout is applied to the input features, each feature is set to zero with probability p during each training iteration. Let's denote the dropout mask as $\mathbf{d}$, where each d_i is either 0 or 1.

The "dropped-out" feature matrix $\mathbf{X}_{\text{d.o.}}$ can be represented as:

$$\mathbf{X}_{\text{d.o.}} = \mathbf{d} \odot \mathbf{X}$$

The objective function $J_{\text{d.o.}}(\mathbf{w})$ when dropout is applied becomes:

$$J_{\text{dropout}}(\mathbf{w}) = \mathbb{E}_{\mathbf{d}} [||\mathbf{y} - \mathbf{w}^T \mathbf{X}_{\text{d.o.}}||^2]$$

Applying dropout to linear regression

Objective Function with Dropout:

$$J_{\text{d.o}}(\mathbf{w}) = \mathbb{E}_{\mathbf{d}} [||\mathbf{y} - \mathbf{w}^T \mathbf{X}_{\text{d.o}}||^2]$$

Expanding the Expectation

$$\begin{aligned} J_{\text{d.o}}(\mathbf{w}) &= \mathbb{E}_{\mathbf{d}} \left[(\mathbf{y} - \mathbf{w}^T \mathbf{X}_{\text{d.o}})(\mathbf{y} - \mathbf{w}^T \mathbf{X}_{\text{d.o}})^T \right] \\ &= \mathbb{E}_{\mathbf{d}} \left[\mathbf{y}\mathbf{y}^T - \mathbf{y}\mathbf{w}^T \mathbf{X}_{\text{d.o}} - \mathbf{X}_{\text{d.o}}\mathbf{w}\mathbf{y}^T + \mathbf{w}^T \mathbf{X}_{\text{d.o}} \mathbf{X}_{\text{d.o}}^T \mathbf{w} \right] \\ &= \mathbf{y}\mathbf{y}^T - 2\mathbf{y}\mathbf{w}^T (1 - p)\mathbf{X} + \mathbf{w}^T (1 - p)\mathbf{X}\mathbf{X}^T \mathbf{w} \\ &= ||\mathbf{y} - \mathbf{w}^T \mathbf{X}||^2 + (1 - p)||\mathbf{w}||^2 \end{aligned}$$

Applying dropout to linear regression

- ▶ The expanded form of the objective function with dropout includes a regularization term $(1 - p)||\mathbf{w}||^2$.
- ▶ This term is similar to the regularization term in ridge regression.

Additional Regularization Techniques

- **L1 Regularization:** Encourages sparsity in the model parameters, often used for feature selection.
- **Mixup:** A data augmentation technique that generates new training examples by interpolating between pairs of examples and their labels.
- **Cutout:** A form of data augmentation where parts of the input data (usually images) are masked out.
- **Gradient Clipping:** Prevents exploding gradients by capping the gradients during backpropagation.